What to show the operator of a free-floating arm: reach maps against rehearsal under a base tilt budget

[authors, order to agree][1]

[1]Nanyang Technological University, Singapore.

Abstract

An arm on a free-floating spacecraft turns the spacecraft when it moves. If the base must stay within a tilt budget, here 2 degrees, only part of the arm's reach can be used, and an operator needs to know which part. That the reach of such an arm depends on the path is known since the early 1990s, and a planning aid that previews a move and checks it before it is executed was proposed in 1992. Reachable sets under a limit on base attitude have been built for a 3 joint arm. What we did not find measured is how far a map drawn for the operator can be trusted under an attitude budget, and we found no study with people. We measure it in simulation for a 7 joint arm under five controllers. One move away from rest, a map pulled in to 0.8 of its length holds from all of 10 starts under the two controllers that ignore the base, and from none of 10 under the three reactionless ones, which reach further (median 0.45 to 0.46 m against 0.37 to 0.39 m). Over sessions of 40 moves no controller keeps it: the whole map still holds in 0 to 4 of 10 sessions, and three sessions end stuck, all under controllers with no pull back to a rest posture. A map recomputed from the current state is right and 12 to 13 s late. A cautious map looked up from 600 stored states needs no simulation and keeps 0.67 of the reach, against 0.29 for one fixed map made as cautious, but it still promises 0.05 m too much on 1.4% of lines, in 43 of 199 test states. The rehearsal of the command itself is the reference for all of this, and in two fenced runs it disagreed with what the arm then did on 1 of 45 and 2 of 46 goals. We describe a tool that runs four displays (the rest map, the map that held, the map from the current state, no map) and a within-subject study protocol that compares them. [study results: no participant has been run]

Keywords: free-floating manipulator, teleoperation, workspace, base attitude, operator feedback

1 Introduction

A manipulator on a free-floating base has no fixed workspace in the usual sense. Momentum is conserved, so every joint motion moves the base, and where the hand ends up depends on the path the joints took [1, 2]. Papadopoulos and Dubowsky [3] made this exact: some points can be reached by some paths only. Reaction null-space control [6, 7] removes the base motion when the arm has joints to spare, but only for part of the reach, and the rest costs base attitude.

In practice the attitude has a budget: an antenna has to stay pointed, a camera has to keep its target. So the question an operator actually has is not "can the arm reach that" but "can the arm reach that with the base still inside its budget". The answer is a region, and the natural aid is to draw it.

We do not claim that this region moves. That follows from path dependence and is thirty years old. We also do not claim the idea of checking a move before making it: Papadopoulos [12] proposed a planner that previews the motion and asks the operator to approve it. What is missing is the size of the problem and what to do about it for the person at the controls. How wrong is a drawn map, under which controller, after how many moves, and is a map worth showing at all next to a check of the command itself? Our contributions:

1. The reach of a free-floating arm under a budget on base tilt, with joint limits and collision with the bus in it, measured for a 7 joint arm under five controllers. A tilt limit during teleoperation is not new: Sato et al. [13] kept one in 1993 by slowing the arm, on a satellite with attitude control. With nothing holding the base, the limit turns into an edge of the reach, and that edge is what we measure.
2. A measured comparison of what can be shown: the map from rest, the same map pulled in, a map recomputed from the current state, a map looked up from stored states, and the rehearsal alone. One move from rest and over sessions of 12 and 40 moves.
3. A finding we did not expect: the map fails first under the controllers built to keep the base still. They reach further, and their failed moves end with joints on their limits, which we counted and did not explain. Plain controllers reach less and keep a pulled-in map for one move, then lose it over a session.
4. A teleoperation tool and a study protocol, written before any data, that compare the displays with people. [and the result]

2 Related work

Workspaces of free-floating arms. Umetani and Yoshida [1] give the generalized Jacobian and already separate a workspace reachable along a straight path from a given posture from one that is guaranteed. Papadopoulos and Dubowsky [3] show that singularities depend on the mass distribution and on the path taken, and define a path-dependent workspace and a path-independent one inside it, where any path works. Vafa and Dubowsky [4] give the virtual manipulator. In all of these the workspace is bounded by singularity or by link lengths, and the base is free to turn. Three lines of work do limit the base. Umetani and Yoshida [19] define five workspaces for a

space arm, one with the attitude constrained and one guaranteed. Jia et al. [18] build reachable sets for a 3 joint arm, among them one that respects a limit on how far the base attitude may deflect. Cocuzza et al. [20] define the zero reaction workspace, where the arm moves with no reaction on the base at all, and later work takes it to arms with many joints [21]. Ours can be read as that workspace with the zero relaxed to a budget. What we add is not the set. It is what happens to the set when it is drawn for an operator: on a 7 joint arm with joint limits, under five controllers, over a session. We know these three lines from abstracts and summaries only. [read Jia 2019, Umetani and Yoshida 2001 and the zero reaction workspace papers in full] For a planar two-link arm we checked our own code against this theory: the band of radii where dynamic singularities exist comes out at 0.41 to 2.36 m on a grid and at the same two numbers from the masses alone (Section 4.5).

A planner for the operator. The closest work to ours is Papadopoulos [12]. The operator names a target and a path. The planner checks whether the path passes through or near a dynamic singularity, previews the simulated motion, and executes after approval. If the path fails it proposes another, built from a detour through the path-independent workspace and small closed motions of the hand that turn the base on purpose. Our rehearsal is this check with a different test in it. The differences are these. The 1992 planner tests for singularity, ours for a tilt budget, joint limits and contact with the bus (collision is named there and left out). It turns the base to get somewhere, we are not allowed to. The example is a planar arm with two joints, so there is no redundancy and no reactionless control. It draws no map, so it does not ask whether a map could replace the check. No operator was tested. We do not propose alternative paths, which that paper does. We read it except for its example section.

Keeping the base still. Torres and Dubowsky [5] map which joint motions disturb the base least. Reaction null-space control [6] and its flight test on ETS-VII [7] remove the disturbance where redundancy allows. Cocuzza et al. [14] minimise the reaction torque by least squares with joint limits inside the solution. Nenchev and Yoshida [23] compare the pseudoinverse of the generalized Jacobian with a null-space controller that keeps the base still, on a 7 joint arm under velocity teleoperation, and find the pseudoinverse the more reliable (from the abstract). These are controllers. They do not tell an operator where the controller will stop working.

A tilt limit while teleoperating. Sato et al. [13] is the nearest work on the budget itself. A ground operator drives a satellite arm, and the satellite attitude has to stay within set limits so the link to the ground holds. Their controller slows the arm as the predicted disturbance grows and feeds the disturbance forward to the attitude control. The operator works in a graphic simulator and, in their words, does so without considering the disturbance. So the limit is hidden from the operator on purpose, and the satellite has attitude control to recover with, which is why slowing down is enough. Ours has none. Slowing a free-floating arm does not change where its base ends up, so the limit cannot be hidden: some goals are simply out. Tested in a simulator, no operators compared. [read from the abstract only, get the paper]

Drift over repeated moves. A redundant arm that brings its hand back does not bring its joints back, and on a free base the attitude does not come back either. Chen et al. [15] plan motions of a free-floating arm so that hand, joints and base

attitude all reset at the end of a task, and review the earlier work on repeatability of redundant arms. That work removes the drift by planning. We measure what the drift does to a displayed map over a session when the operator picks the goals and nobody plans against it. Our finding that sessions without a pull to rest posture get stuck may be this known drift seen from the operator's seat. We have not shown that it is.

Showing limits to operators. Virtual fixtures [8, 9] put limits in front of users, and have user studies, but the limits are fixed in space. Coelho et al. [16] give the operator of an aerial manipulator a haptic wall at the limits of the null-space motion of the base. That wall moves with the state, the base is actuated, and the test is a numerical simulation with no users. Hauck et al. [24] draw the reachable space for people planning robot motions in virtual reality and test it with 20 of them, for an arm on a fixed base. Nearest to us, Pietras et al. [22] describe ways to support a ground operator against breaking limits on base attitude, such as antenna pointing, on a real-time simulator meant for tests with an operator in the loop. We have the abstract only, and it reports no test with people. [get the Pietras paper: if it tested operators, the last claim of the gap changes]

The gap. Path dependence is known, a check before execution is known, a tilt limit in teleoperation is known, joint drift is known, and reachable sets under a limit on base attitude exist for a 3 joint arm [18]. We found no work that draws such a set for the operator of a base that nothing holds and measures how far it can be trusted across controllers and over a session, and we found no study with people. This is what about 80 searches turned up, with one paper read almost in full and the rest from abstracts and excerpts, not a proof that nobody did it. Japanese and Chinese work and the conference papers of the 1990s are thinly covered. [one systematic search before submission, with forward citations]

3 System

A free-floating bus carries a 7 joint arm in MuJoCo [10]. Nothing holds the bus: no thrusters, no reaction wheels. The hand is driven by resolved-rate control on the generalized Jacobian, and the spare joint is used to cancel base reaction where it can. Base tilt is counted from the attitude at the rest pose, and the budget is 2 degrees. The bus is a 1 m cube of 300 kg. The arm weighs 12.5 kg in seven links and measures 0.97 m from shoulder to hand at full stretch. Joint rates are capped at 2 rad/s. The stacked hand and base rows are inverted with a damping of 10^{-6}, which was never tuned and decides where some lines end (see the paragraph on joint limits inside the controller). [link lengths and gains: table from the CA3 paper]

The reach is probed along 98 directions from the rest position of the hand. Along each one the hand is driven out until the tilt reaches the budget, the arm would hit the bus, or the hand stalls. The 98 end points form the map the operator sees, which we call the rest map. Distance along a direction is given as a fraction of the rest map, so 1 is its edge.

A command is checked by rehearsal: the move is simulated from the state the arm is really in, and refused if it ends outside the budget. This is the check of the 1992 planner [12] with the budget, the joint limits and the bus as the test. Every result in

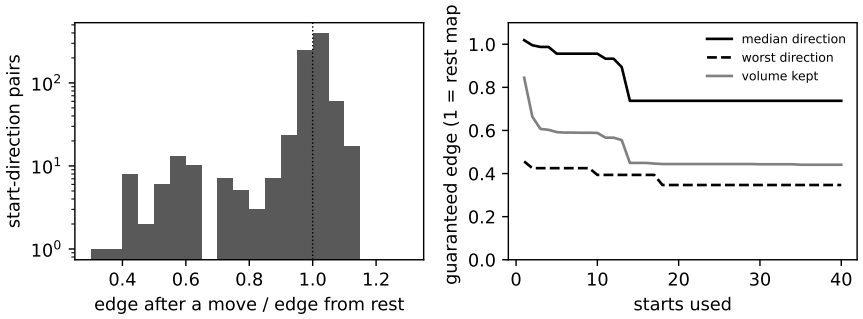


Fig. 1 Left: edge after one move, as a fraction of the edge from rest, 800 start and direction pairs. Most are at 1, a tail collapses. Right: the edge that holds from every start used so far, against the number of starts. Scripted runs.

this paper is scored against it, so here it is right by construction. Against what the arm then did it was off on 1 of 45 and 2 of 46 goals in two fenced runs, and in the stored sessions 4 of 826 accepted moves ended over the budget, by 0.013 degrees at most. It takes about 4 s for a far goal.

4 The edge moves

4.1 Measurement

We moved the hand to a random start one move away from rest, then found by bisection how far it could go along each of 20 random directions before the budget stopped it, and compared with the same search from rest. 40 starts, 800 start and direction pairs.

In 80% of pairs the edge is within 0.05 of the rest map. In 7% it is below 0.8, and the worst is 0.35 (Fig. 1, left). The bad pairs are not a corner of the space: 17 of the 40 starts and 13 of the 20 directions have at least one. They are more common on moves across the start direction (8 to 10% against 3 to 4% along it) and when the start left more tilt behind (15% above 0.8 degrees against 5% below 0.4), but neither tells you which pair will fail.

So for random goals near its edge a map drawn from rest is wrong about one time in fourteen, and neither depth nor tilt left says which one.

4.2 Two explanations that failed

Leftover tilt. The tilt is counted from rest, so a move that leaves tilt behind has less budget for the next one. If that were the cause, the right map would be the rest map with the budget corrected. We repeated the search with the leftover tilt forgiven. The edge did not go back: the median distance to the rest edge was 0.05 with tilt forgiven against 0.03 with it kept. Tilt is a vector and the budget is a ball around the rest attitude, so leftover tilt moves the edge outward in some directions and inward in others.

Enclosed area. For a free-floating system the attitude change over a closed path grows with the area the path encloses. Large shifts do only happen at large area (worst -0.29 m above 0.09 m^2, -0.05 m below 0.03 m^2), but a linear fit on signed area explains 7% of the shift.

We do not have a formula for the edge. What we have is the simulation.

4.3 Whose edge it is

Everything above is under one controller, ours: impedance on the hand with the spare joints used to cancel base reaction. We repeated it under three others: damped least squares and null-space impedance, which ignore the base, and reaction null-space control without impedance. 10 starts one move from rest, each controller against its own rest map on 98 directions.

The two reactionless controllers reach 0.24 to 0.75 m from rest, median 0.45. The two plain ones reach 0.24 to 0.64 m, median 0.37 to 0.39. Pulled in to 0.8, the map of a plain controller holds for every direction from all 10 starts. The map of a reactionless one holds for 90% of start and direction pairs and for every direction from none of the 10 starts. At 0.95 the plain maps hold for 86 to 89% of pairs, so their edge moves too, by less.

A reactionless controller arrives with less tilt spent (median 0.40 degrees against 1.07) and with joints on their limits: its failed moves end with a median of 2 joints at a limit, those of the plain controllers with none. So the far reach is bought with joint range, and what is left of the range depends on the start. This is read off counts, we have not shown the mechanism.

Between 11 and 26 of the 98 lines end in a stall, under every controller. These are not a singularity of the free base. The arm has run out of length. Under our controller all 26 end with the hand 0.96 to 0.97 m from the shoulder, where full stretch is 0.97 m, and with elbow and wrist within 12 degrees of straight. A bus ten or a thousand times heavier leaves the stall where it is, a hand point 0.10 m further out moves it by 0.10 m, and the other controllers stop at the same distance. The reach on these lines is 0.24 to 0.53 m, median 0.29: in those directions the rest pose is close to full stretch. Two things blur this. The last 2 to 14 cm are a crawl the controller makes, because near full stretch it asks for 5 to 15 rad/s and is scaled down to the 2 rad/s cap. And not every stall is this one: on 10 of the 26 lines one joint also sits on its limit, and from two other states 4 of 34 stalls were joint-limit stalls with the arm bent. The map should tell the two apart and does not yet.

Joint limits inside the controller. The reactionless controllers lose the map with joints sitting on their limits, so we tried one that looks at the limits while it solves, after Cocuzza et al. [14]: our controller with a weighted least-norm solve [17], where a joint heading for its limit costs more the closer it gets. It changed almost nothing. Rest reach 0.24 to 0.75 m, median 0.46, and two of the 98 lines move, by 0.08 and 0.09 m. One move out, the 0.8 map holds for 91% of pairs and from 0 of 10 starts, against 90% and 0 of 10 without it. Over 40 moves it ends at 90% of pairs and 0 of 10 sessions, with 23 of 400 moves refused against 15 without it. Our first reason for this was wrong. We wrote that the hand takes three joints, a level base three, and one of seven is left to spare the others with. The count is right and the conclusion is

not. At rest the one free direction is the roll of the tool, which moves neither the hand nor the base and can take no load off another joint. To stop the joint nearest its limit through that direction would need a median of 3400 rad/s against a cap of 2. With the damping in the solve switched off, the weights slow joints 1 to 6 by under 0.1%. What they change at our damping comes through directions the damped inverse has already given up. So the result belongs to that damping: at 10^{-8} the plain controller stalls at 0.40 m on a line where the weighted one reaches 0.73 m, and at our 10^{-6} the two agree to 1 mm. This is one velocity-level version, not their solver, and while the base is held level it has nothing to work with.

We then let the base go a little: the same weights, with the three base rows a cost and not a constraint, and up to 1.5 degrees of the budget to spend on joint room. It is a different trade, not a better one. On the same 392 goals one move out, from 4 starts, it reached 372 and our controller 373. On 89 of the lines it gained 9 goals. On the other 9 its own reach from rest is 10 to 41 cm shorter and it lost 10. Its rest map is smaller, mean reach 0.456 against 0.474 m. Four starts and first values, nothing tuned, and one start carries 6 of the 9.

4.4 Long sessions

An operator does not make one move. We ran sessions of 40 moves, 10 per controller. Each goal is 0.2 to 0.8 of the way to the rest edge along a random direction, rehearsed, and made only if the rehearsal says yes.

In a first run of 12 moves, 10 sessions per controller, the map pulled in to 0.8 holds after move 12 for 91% of directions under our controller, 80% under reaction null space, 93% under damped least squares and, rounded, 100% under null-space impedance. Every direction holds in 0, 0, 4 and 9 of the 10 sessions.

After move 40 the map pulled in to 0.8 holds for 88% of directions under our controller, 83% under reaction null space, 65% under damped least squares and 85% under null-space impedance. Every direction holds in 1, 1, 3 and 4 of the 10 sessions. Null-space impedance held everything in 8 of 10 at move 20, so the loss comes late. Of the 400 goals, all well inside the map, the rehearsal refused 15, 46, 40 and 3. Two sessions under damped least squares and one under reaction null space ended with the last 10 goals all refused. Neither has a pull back to a rest posture. The controllers that have one did not get stuck. Tilt left behind rises over the first moves and levels off (median over the first 10 moves against the last 10: 0.31 and 0.45 degrees for ours, 0.95 and 1.22 for damped least squares). The stuck sessions do not fit that. They sit at a median of 2.01 degrees, on the budget, and all 30 of their refusals are for tilt. A refused move is not made, so nothing brings the base back and the next goal is refused too. Whether drift in posture is what took them there we have not shown.

That a redundant arm drifts in posture over repeated moves is known [15]. What is new here is what it costs the map.

So no controller here keeps a fixed map over a session at 0.8 of its length. The rehearsal is what all of this is scored against, and we expect it to be the thing an operator can lean on. That is a prediction. No operator was tested.

4.5 A planar check

To tie the code to the known theory we built the textbook system separately: a planar arm with two links on a free base, exact momentum, no physics engine, on made-up numbers (base 200 kg, two links of 10 kg and 1 m). With no tilt limit, the radii at which a dynamic singularity exists run from 0.41 to 2.36 m on a grid of joint angles. The same band from the masses and lengths alone, as the sum and difference of the three barycentric lengths, is 0.41 to 2.36 m.

5 What to show instead

5.1 A map that held from every start

Take, for each direction, the worst edge over all starts. After 40 starts the median direction keeps 0.74 of the rest edge and the worst keeps 0.35, which is 44% of the rest reach by volume. The figure was the same at 20 starts as at 40 (Fig. 1, right), so for starts one move from rest it has probably stopped shrinking. It is not a guarantee, and a second move shows it. From 40 starts two moves away from rest, one had already spent the budget on the way. From 24 of the other 39 the rehearsal refused at least one tip of this map, 4.3% of all pairs, on 70 of the 98 directions. Pulling every direction in to 0.8 of its length leaves 11 starts and 0.9% of pairs. A fixed map cannot be made safe by shrinking it against sampled starts. Measured again on the 98 directions of the map itself, same 40 starts, the map that held keeps 0.67 of the rest edge on the median direction, 0.24 on the worst, and 40% of the volume. 28 of the 98 directions keep less than half and 5 keep everything.

5.2 A map from where the arm is

Run the 98 probes again from the current state. On one thread this takes 85 s. Shared over 10 processes it takes 13.0 s from rest and 12.2 s after a detour, on one laptop. So this map is right, for a hand that has stopped, and 12 to 13 s old. The tool shows a yellow marker on the hand while the map is old and a green one when it is current.

5.3 A map looked up

The map from the current state can be stored instead of computed. We took 200 states, one and two moves from rest, and for each the 98 reaches from a fast model without physics. Against the simulation on 199 states that model is 9 mm off on the median, within 0.056 m on 95% of lines and 0.54 m off at worst. A new state is given the weighted average of its 5 nearest stored states. On states left out, the median error is 0.034 m, against 0.120 m for the average of all maps (an average, not the rest map the tool draws). Against the simulated map our first figure came from 5 states: 0.05 m too much on 4.9% of lines. On 101 to 125 simulated states it is 14 to 16% of lines too far by 0.05 m and about 20% too short, median error 0.033 to 0.035 m. So the plain average is too far on about one line in seven, the same failure as the fixed map. It needs no simulation when it answers. We have not timed it.

Made careful. A map that promises too much is the costly kind of wrong, so we made the lookup cautious: per direction the shortest reach among the k nearest stored states, pulled in by a factor, and no reach at all when the base is already over the budget. We scored it against two fixed maps given the same allowance: per direction a low percentile of the stored reaches, and the same shrunk by the share of the tilt budget already spent. The second is the fair one, because the lookup's neighbours carry the tilt for free. 1000 states, one or two moves from rest, 600 stored, 200 to pick the settings on (the furthest-reaching setting that promises 0.05 m too much on under 1% of state and direction pairs), 200 to score, and the scoring states simulated in full. Two of them start with the arm inside the clearance around the bus, where the tool would refuse the move, and are left out.

Against the simulation on 199 states the lookup (10 nearest, pulled in to 0.9) keeps 0.67 of the total reach and promises 0.05 m too much on 1.4% of pairs (0.8 to 2.1 over states). The fixed map (2nd percentile) keeps 0.29 and is over on 0.7% (0.1 to 1.3). The fixed map shrunk with tilt keeps 0.39 and is over on 0.6% (0.3 to 0.9). So the lookup shows 2.3 times the reach of a fixed map and 1.7 times that of the fair one. It does not do so at the same risk. Its misses are spread over 43 of the 199 states, against 9 and 25, and its worst miss is 0.44 m against 0.16 and 0.22 m.

The lookup is only as good as its nearest stored state. The figures in this paragraph are on the fast estimate, not the simulation. Split by distance to the nearest stored state, the furthest quarter of the scoring states is over on 5.0% of pairs, about the same as the fixed map (4.9%), and the nearest quarter on 0.1%. Held out by region, 20 times, the lookup is over on a median of 6.3% against 1.6% for the fixed map, and worse in 19 of the 20. Falling back to the fixed map beyond a set distance halves that and does not close it. So it holds for states drawn like the stored ones and not outside them. Whether a setting keeps the 1% it was picked for is the luck of the split: over 100 random splits the lookup averaged 0.85% and the fixed map 0.75%, each above 1% in about three of ten. With 200 stored states in place of 600 the lookup kept 0.56. A fixed map that holds from every stored state keeps 0.09 of the reach, once the states with no reach at all are set aside.

5.4 No map

Draw nothing. Rehearse every command and refuse the ones that would break the budget. This is as right as the rehearsal and gives the operator nothing to plan with: we expect them to find the edge by being refused.

6 Study

6.1 Design

Question. The rest map promises reach that is gone once the arm has moved. Does that cost an operator commands that fail, and does a map drawn from where the arm is remove them when it arrives 12 to 13 s late?

Hypotheses, written before data, each with the direction the simulation gives. H1, the main one: more test goals have a command the fence had to bring back with

the rest map than with the map from the current state. In simulation the rest map allows a goal the arm cannot reach on 8% of pairs with the goal at 0.5 to 1.0 of its edge and on 13% at 0.8 to 1.0, and the current-state map is right once it is current. H2: seconds per test goal are lower with no map than with the rest map. A scripted operator took 22.1 s against 32.6 s on the goals as the tool draws them. Its times by class, put together for the goals of this design, give about 19 s against 27 s. That gap is the two traps, which took the rule 44 s each because it gives up only 40 s after a command. Had it given up when the fence had brought the arm back, about 15 s, the gap would be about 3 s. H3: on test goals that are reached, seconds are higher with the current-state map than with the rest map, by about the age of the map. The scripted operator took 20.4 s against 4.5 s (medians). The scripted operator is a rule, not a person. It sends once when the dot is on the goal and always waits for a map to be current, so its numbers say what the task can show and not what people do. H1 and H2 need people to send on a goal the rest map allows. H1 and H3 need them to wait for the current-state map. So these are not three separate pieces of evidence: if people do not wait, H1 and H3 fail together. We predict no direction for workload or trust: the rule has neither.

Conditions. Within subject, four conditions, one block each: the rest map (the control), the map that held, the map from the current state, and no map. In the three map conditions the dot cannot be put outside the map on screen. With the current-state map the rest map stays on screen and the hand marker is yellow until the new map is in. With no map every command is rehearsed first, about 4 s for a far goal, and refused if it would fail. Before a block the participant is told what the display is, in the same words for everyone. Rest map: it was drawn with the arm at rest and can be wrong once the arm has moved. Map that held: it held from every start we tried one move from rest and is smaller than what the arm can reach. Current-state map: it is drawn from where the arm is now and takes about 12 s, and until the marker turns green the screen still shows the rest map. No map: every command is checked before it runs and refused if it would fail. Every session runs on the same PC with 10 processes for the current-state map, because its delay depends on the machine.

Two results are fixed by the tool and are not hypotheses. No map cannot give a command the fence brings back, because the rehearsal refuses it, and the map that held should not, because every pair in the pool is checked against it. A reachable goal outside the map that held has to be given up, because the dot cannot go there. We report both as checks that the tool did what it should. The block with the map that held therefore carries no hypothesis. It is there for the ratings and as a level to read the others against.

Task and trial. Put the hand on the goal, or give it up as out of reach. One trial is one pair of goals. The lead goal lies well inside every map, moves the arm off rest and is not scored. The test goal is the scored one and starts from where the lead goal left the arm. After it the arm goes back to rest. A goal is reached when the hand has stayed within 2 cm of it for 0.5 s. Whether a test goal can be reached is rehearsed from the arm's real state when it appears, and the participant does not see the answer.

A *wrong go* is a sent command that the fence brings back: the base passes 2 deg, the arm comes within 2 cm of the bus, or the hand is still short after 30 s. The arm

returns to where the command started and the participant may try again or give up. A command refused under no map is not a wrong go and is counted on its own. A *give-up* is the participant pressing the give-up key. It ends the goal and adds 30 s to its time. A test goal still open after 90 s ends, counts as given up and takes the 30 s as well. A *missed go* is a test goal that the rehearsal says can be reached and that ends given up. Giving up a goal that cannot be reached is the right answer.

What the participant is told to do. One aim: the lowest total time over the block. They are told what things cost. A give-up adds 30 s. A command the fence brings back adds nothing but the time it took, and the arm is back where it was. An earlier draft told two aims in order, no command brought back and then speed. We dropped that for three reasons. An order with no price leaves each participant to set the price, and one who takes it at its word never sends near the edge of the rest map: that scores well on wrong goes without doing the task. With the fence a wrong go costs time and no attitude, so the order says something our own simulation does not support. And it was chosen because a rule that is only fast gains nothing by waiting, which is what H1 is there to find out. With time as the only aim a give-up has a price, never sending is slow, and whether a late map is worth the wait is left to the participant. On the scripted times by class, sending on every goal the rest map allows costs about 27 s per test goal, or about 22 s for a rule that gives up as soon as the fence has brought the arm back. Never sending outside the map that held costs about 35 s, waiting for the current-state map on every goal about 33 s, and waiting only outside the map that held about 29 s. So for the rule holding back loses, and waiting loses 2 to 7 s a test goal. A person can steer while the map is drawn and the rule cannot, so this is not a forecast. But by our own arithmetic H1 may well fail under this score, and a result for H1 holds for this score.

Goal sets. As the tool stands, the 15 goals of a block are spread from 0.5 to 1.4 of the rest edge and 7 are scored. In the scripted run 47 of 112 scored goals lay beyond 1.05 of the rest edge, where every map says no, and only 3 were goals that the rest map allows and the arm cannot reach. We call such a pair a trap. As built, a block holds a trap about one time in five, and the conditions can hardly differ. So the sets are drawn again before the first participant. We draw pairs with a fixed seed, rehearse each from the state its lead goal leaves, and sort them into four classes. Traps: the test goal is inside the rest map, outside the map that held, and cannot be reached. Edge goals: inside the rest map, outside the map that held, reachable. Inner goals: inside the map that held, reachable. Outer goals: at 1.2 to 1.4 of the rest edge, not reachable. A person's lead move does not end where the rehearsal's did, so a pair is kept only if its class stays the same when the lead move ends 2 cm off the lead goal in each of six directions. A block has 10 pairs: 2 traps, 4 edge, 2 inner, 2 outer. There are as many sets as conditions. Every set has the same number of pairs in each class and the same spread of depth within a class, and no pair is in two sets. Within a set the edge goals are drawn at the depths of the traps, one within 0.03 of each trap. In the map data traps lie further out than reachable goals of the same shell (28% of pairs are traps at 0.95 to 1.0 of the rest edge, 7 to 9% below 0.8), and the dot's colour shows depth. Without the match a participant could sort traps from edge goals by colour under any map. A participant meets each set once and so sees no pair twice.

Which set goes with which condition moves on by one with each participant and by one more after every four. As the tool does it now, two sets always come first or last in a session and two always in the middle. With the extra step each set comes at each place 4 to 6 times in 20 sessions and each condition meets each set 4 to 6 times. This is how the map and not the goal differs between conditions: the sets are matched class by class, and what is left of a set is spread over the conditions and the places. Pairs come in a new random order for each participant and block.

Four things about these sets are choices. Two traps among the eight test goals inside the rest map is about three times what the maps give (6 to 8% of such pairs), so the study measures what a trap costs and not how often one comes, and the rest map looks worse here than it is. No test goal lies just outside the rest map, so the study does not measure what the rest map loses by showing too little. Depth tells a participant nothing about a trap here, and in the maps it tells a little. And the 2 cm check does not cover a lead move that took another path, so a planted trap may still not be one for a person: scoring uses the rehearsal from their own state, we report how many traps each participant met in each block, and the sample size below does not count on all of them.

Outcomes, on test goals only. Primary: the number of test goals in a block with at least one wrong go. The participant's aim is time and ours is a count. That is on purpose: the count is what the map on screen leads to, and time is what the participant weighs it against. Secondary: missed goes on edge goals; seconds per test goal with the 30 s per give-up and the rehearsal time in it, and again without the 30 s, which is our choice and not a cost of the mission; raw NASA-TLX [11], the mean of six scales from 0 to 100, after each block; and one trust rating after each block, "How much did you trust what the screen showed", from 1 (not at all) to 7 (fully). We also report, with no test: of the traps a participant met, the share they sent on with the rest map and the share they sent on before the current-state map was in; how long they waited for that map; missed goes by class; refused commands; commands per goal; and worst tilt. Seconds per goal was the primary measure in the protocol. We moved it because in the scripted run it follows the number of goals a display makes the operator give up, at 30 s each, more than it follows the map.

Order. A Williams square: four orders, five participants each, every condition once in every place and once after every other. This balances what one block leaves in the next only if that is the same whatever comes next. It may not be: a participant who has met the current-state map may send less with the rest map afterwards. We cannot remove that. We show it: the H1 difference for the 10 who had the rest map before the current-state map next to the 10 who had it after, with no test.

Training, about 10 min. The experimenter reads a fixed sheet and shows the controls. The sheet says what the fence does and what a command brought back and a give-up cost. All participants use the keyboard. Then four practice pairs under the display of the participant's own first block: two inner, one edge and one outer. An earlier draft gave these four with the rest map to everyone, with a trap among them. That gave the control more practice than any other display and showed it failing before the first block. Practice under the first display is spread over the conditions by the orders. A participant goes on when they have reached an inner test goal and given

up the outer one. The four pairs may be repeated once. Each block then starts with two practice pairs under its own condition, one edge and one trap in random order, which are not scored. So every display has been met on the two kinds of goal where the displays differ before it is scored, and under the two displays that allow a wrong go the participant has had the chance to see one. Practice pairs are in no set. During a block the experimenter says nothing.

A session is then 12 practice pairs and 40 scored pairs. The hour is spent like this: 5 min for consent and the sheet, 8 min for the controls and the four practice pairs, four blocks of 1 min of fixed words, 12 pairs and 2 min of ratings, and 3 min to close. That is 60 min if a pair takes 40 s on the clock, which is our guess. The scripted rule needs 9 to 23 simulated seconds a pair and does not hesitate. On top come the stop at every goal while the tool rehearses the truth, which no time column holds, and the time people take to think. Two pilot sessions check the clock and are not data. If either runs over 65 min, every block is cut to 8 scored pairs, 2 traps, 3 edge, 1 inner and 2 outer, before the first participant. Traps and edge goals are the last to be cut.

Exclusions, fixed now. A participant is left out and replaced in the same order if they do not pass the practice after the repeat, if they do not finish all blocks, if fewer than 8 pairs of any block can be scored, or if the tool or the PC fails during a block, which includes a current-state map that takes more than 20 s. A pair is not scored if its lead goal was not reached, because the test goal then starts somewhere else. Nobody and no pair is left out for being slow, for wrong goes or for giving up. A participant who gives up goals that every display allows stays in, and we report inner goals given up per condition so that it shows. We report how many were left out and why.

Analysis. The unit is the participant. Pairs are nested in participants and are not independent: what a trap did to a person changes what they do on the next. So no test counts pairs. Every outcome is one number per participant and condition: a count for wrong goes and for missed goes, the mean for seconds per test goal, the median for seconds on reached test goals, the mean of the six scales, the rating. Two conditions are compared within participant by the exact Wilcoxon signed-rank test, with zero differences dropped and tied ones sharing a rank. We fit no logistic model to the wrong goes: two conditions have none by construction and a third may have none, and such a model does not settle on empty cells. The primary outcome is coarse. With two traps a block it is 0, 1 or 2 for most participants and many differences will be zero. The test and the simulation below both have that in them.

There is one confirmatory test, H1, two-sided at 0.05. Five secondary tests share 0.05 by Holm's method: H2, H3, missed goes on edge goals with the rest map against the current-state map, a Friedman test over the conditions for workload, and one for trust. The missed goes have no predicted direction: the rule misses no edge goal under either map. An earlier draft left the Friedman tests outside the correction and had one on all missed goes. The goal set decides that one, since the map that held forces four a block, so it is gone. Nothing else is tested. With each test we give the mean difference within participant, a 95% interval from 10000 resamples of participants, and how many participants went each way. H3 is over reached goals only, and which goals are reached is partly the participant's choice, so the number reached goes next to it.

Under the rest map and under no map nothing on screen stops a reachable test goal, so a missed go there is the participant's and not the display's. We split those into goals given up with no command sent and goals given up after one, and report them apart from the ones the map that held forces. One reading is fixed now. No difference in wrong goes together with more missed edge goals under the rest map is reach lost to the rest map. It is not support for H1 and it is not a sign that the rest map is safe. We also report every outcome by place in the session and by goal set, to see what order and set have left. We do not look at the data before all participants are in.

What would count against us. If the rest map has no more wrong goes than the current-state map, H1 fails, and the data say which half failed. Few traps sent on with the rest map: people held back from its edge, and the missed edge goals show what that cost. Many sent on before the current-state map was in: people did not wait, and a map that is 12 s late is then not used as a map. With 20 participants a failed H1 is weak evidence that the maps do not differ unless one of these two is plain in the data (Table 1). If no map is not faster than the rest map, H2 fails. If no map is the fastest, has no wrong goes and is rated no more work, which the scripted run predicts for the time, then for one goal at a time a map is not worth showing and the paper has to say so. The task hands over one goal at a time, so it can show what a map costs and cannot show what a map is for, which is planning.

Sample size. Human rates are unknown, so we can only say what the design can see under assumptions we state. A wrong go needs a trap, and on a trap a participant either sends or does not. Let a be the chance that they send with the rest map, and b the chance that they send before the current-state map is in. The scripted rule has $a = 1$ and $b = 0$. We simulated the study with participants who differ by a standard deviation of 1 on the logit scale and applied the test above (Table 1). The upper rows are the kind case: every planted trap is a trap for the participant, a person leans the same way under both maps, and no other goal gives a wrong go. Then 20 participants find H1 in 93% of simulated studies if $a = 0.9$ and half do not wait, and in 72% if $a = 0.7$ and $b = 0.35$. Each of the three flatters us. One at a time on that last case: a planted trap that is one for the person four times in five gives 50%, leans that are not linked 61%, a wrong go on one other goal in twenty 52%. Two more things we cannot rule out cost as much: no second trap sent on after a wrong go in the same block gives 42%, and half the participants sending at 0.4 with the rest map because they met the current-state map first gives 26%. The lower rows take the first three together. Then 20 participants find H1 in 93% if $a = 0.9$ and $b = 0.2$, in 47% if half do not wait, and in 32% if $a = 0.7$ and $b = 0.35$, and 30 participants give 99%, 68% and 48%. A third trap a block would lift the 32% to 50%, at four times the rate the maps give, and we keep two. So 20 is enough only if most people send with the rest map and most wait for the current one. With time as the only aim we do not know that they will wait, and for the rule waiting does not pay. We plan for 20, which fills the orders evenly, and we print this table next to the result. One trap a block, nearer what the maps give, finds $a = 0.7$ and $b = 0.35$ in 38% in the kind case. With the goals as the tool draws them now and participants who do exactly as the rule, 20 find H1 in 11%.

With the looked-up map as a fifth condition. A block has 8 pairs, 2 traps, 3 edge, 1 inner and 2 outer, in five sets, so a session keeps 40 scored pairs and adds

Table 1 Share of 4000 simulated studies in which the rest map has more wrong goes than the current-state map at $p < 0.05$. a, b: chance that a participant sends on a trap with the rest map, and before the current-state map is in. Rest, live: the wrong goes a block that the row then gives under the two maps. All of these are assumptions, not data.

goals	a	b	rest	live	$n = 12$	$n = 20$	$n = 30$
Every planted trap met, leans linked, no other wrong go							
this design	1.0	0.0	2.00	0.00	1.00	1.00	1.00
this design	0.9	0.2	1.73	0.48	0.99	1.00	1.00
this design	0.9	0.5	1.73	1.00	0.64	0.93	0.99
this design	0.7	0.35	1.34	0.75	0.40	0.72	0.93
one trap a block	0.7	0.35	0.67	0.37	0.15	0.38	0.60
as the tool draws them now	1.0	0.0	0.19	0.00	0.01	0.11	0.42
A planted trap met 4 times in 5, leans not linked, a wrong go on 1 other goal in 20							
this design	1.0	0.0	1.92	0.32	0.99	1.00	1.00
this design	0.9	0.2	1.70	0.70	0.69	0.93	0.99
this design	0.9	0.5	1.70	1.12	0.25	0.47	0.68
this design	0.9	0.7	1.70	1.39	0.09	0.17	0.24
this design	0.7	0.35	1.39	0.92	0.17	0.32	0.48
three traps planted	0.7	0.35	1.88	1.18	0.28	0.50	0.71
this design	0.5	0.2	1.12	0.70	0.13	0.27	0.43

two practice pairs and one round of fixed words and ratings. That does not fit the hour: the same budget gives 64 min, or 61 min with 7 pairs a block and one outer goal. A Williams design for five conditions has ten orders, two participants each, and each condition meets each set 4 times. A participant is left out if fewer than 6 pairs of a block can be scored. H1, its two blocks and its traps stay as they are, so the table does not change but for a little less noise in its lower rows, where 6 and not 8 test goals lie inside the rest map. Two secondary contrasts join: fewer wrong goes with the looked-up map than with the rest map, and fewer seconds on reached test goals than with the current-state map, because it answers at once. Holm's method then covers seven tests and not five. What we give up is one edge and one inner pair per block, which makes missed goes and seconds coarser in every condition, and the traps are then 2 of the 6 test goals inside the rest map. Against the full simulation the looked-up map keeps 0.67 of the reach and promises 0.05 m too much on 1.4% of pairs, in 43 of 199 states, and only near its stored states. So which traps it lets through has to be counted by rehearsal when the pool is drawn. [The tool runs the looked-up map as a fifth block now, in ten orders, but with four goal sets, so the rest map and the looked-up map share one (D60). Four or five conditions: for the team to decide once the pilot has timed a pair.]

[Tool and script, before the first participant: (1) a script that draws and sorts the pairs, with the 2 cm check and the depth match, and teleop.py taking that list where it now takes data/decide4_medium.npz and scores a random 7 of 15 goals; (2) the block prompt says "Fastest total wins": add what a give-up and a command brought back cost; (3) log for every command whether the current-state map was in, and how long each map took; (4) a time-out is logged as not given up and without the 30 s; (5) the gamepad buttons are untested, which is why the keyboard; (6) feedback.py tests

seconds per goal first and all pairs, by another form of the signed-rank test, and sums wrong goes: make it H1 first, the planned tests only, the exact test of power.py, and test goals with at least one wrong go; (7) the set index in teleop.py gets one more step after every four participants; (8) practice pairs under the first block's display, and one edge and one trap before each block; (9) run the scripted operator through the new sets and report it as scripted. Also find out why the scripted operator gave up all 4 reachable goals just outside the rest map under the current-state map without sending.]

[For the team: time as the only aim follows D28. If a command brought back should cost more than its time, give it a price in seconds and put it in the prompt. Not an order of aims with no price.]

[Ethics approval, consent form and recruitment of the 20 students: for the team to settle before the first participant.]

6.2 Results

[No participant has been run. Ethics approval and consent to settle first.] A scripted operator has been run through the four blocks of the first design, 28 scored goals per display, on the goals as the tool draws them now. It is a rule and not a person, and nothing here is participant data. It took 32.6 s per goal with the rest map, 34.7 with the map that held, 34.9 with the map from the current state and 22.1 with no map, and 15 of its sends were refused.

7 Limits

Simulation only. One arm, one bus mass, one budget. The controller comparison and the sessions are 10 starts or 10 sessions per controller. No recovery move was tried on a stuck session. The map that held is tested on one-move starts. The delay of the current-state map is the delay on one laptop. The reach is probed on 98 lines from one point, so the map is a set of lines and not a volume with a proof behind it. The damping in the controller's solve was never tuned and moves where some lines end. The looked-up map holds near its stored states and not outside them. No contact, no flexible parts, no sensor noise.

References

[1] Y. Umetani and K. Yoshida, "Resolved motion rate control of space manipulators with generalized Jacobian matrix," *IEEE Trans. Robot. Autom.*, vol. 5, no. 3, pp. 303–314, Jun. 1989, doi:10.1109/70.34766.

[2] S. Dubowsky and E. Papadopoulos, "The kinematics, dynamics, and control of free-flying and free-floating space robotic systems," *IEEE Trans. Robot. Autom.*, vol. 9, no. 5, pp. 531–543, Oct. 1993.

[3] E. Papadopoulos and S. Dubowsky, "Dynamic singularities in free-floating space manipulators," *ASME J. Dyn. Syst. Meas. Control*, vol. 115, no. 1, pp. 44–52, Mar. 1993.

[4] Z. Vafa and S. Dubowsky, "The kinematics and dynamics of space manipulators: the virtual manipulator approach," *Int. J. Robot. Res.*, vol. 9, no. 4, pp. 3–21, Aug. 1990.

[5] M. A. Torres and S. Dubowsky, "Minimizing spacecraft attitude disturbances in space manipulator systems," *J. Guid. Control Dyn.*, vol. 15, no. 4, pp. 1010–1017, 1992, doi:10.2514/3.20936.

[6] D. N. Nenchev, K. Yoshida, P. Vichitkulsawat, and M. Uchiyama, "Reaction null-space control of flexible structure mounted manipulator systems," *IEEE Trans. Robot. Autom.*, vol. 15, no. 6, pp. 1011–1023, Dec. 1999.

[7] K. Yoshida, K. Hashizume, and S. Abiko, "Zero reaction maneuver: flight validation with ETS-VII space robot and extension to kinematically redundant arm," in *Proc. IEEE ICRA*, Seoul, May 2001, pp. 441–446, doi:10.1109/ROBOT.2001.932590.

[8] L. B. Rosenberg, "Virtual fixtures: perceptual tools for telerobotic manipulation," in *Proc. IEEE Virtual Reality Annu. Int. Symp.*, Seattle, Sep. 1993, pp. 76–82, doi:10.1109/VRAIS.1993.380795.

[9] J. J. Abbott, P. Marayong, and A. M. Okamura, "Haptic virtual fixtures for robot-assisted manipulation," in *Robotics Research*, Springer Tracts in Advanced Robotics, vol. 28, Springer, 2007, pp. 49–64.

[10] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: a physics engine for model-based control," in *Proc. IEEE/RSJ IROS*, Vilamoura, Oct. 2012, pp. 5026–5033, doi:10.1109/IROS.2012.6386109.

[11] S. G. Hart and L. E. Staveland, "Development of NASA-TLX: results of empirical and theoretical research," in *Human Mental Workload*, Advances in Psychology, vol. 52, North-Holland, 1988, pp. 139–183, doi:10.1016/S0166-4115(08)62386-9.

[12] E. Papadopoulos, "Teleoperation of free-floating space manipulator systems," in *Proc. SPIE, Telemanipulator Technology*, 1992. [volume and pages]

[13] Y. Sato, M. Hirata, F. Nagashima, T. Maruyama, and T. Uchiyama, "Reducing attitude disturbances while teleoperating a space manipulator," in *Proc. IEEE ICRA*, vol. 3, 1993, pp. 516–523.

[14] S. Cocuzza, I. Pretto, and S. Debei, "Novel reaction control techniques for redundant space manipulators: theory and simulated microgravity tests," *Acta Astronaut.*, vol. 68, no. 11–12, pp. 1712–1721, 2011, doi:10.1016/j.actaastro.2010.06.014.

[15] G. Chen, L. Zhang, Q. Jia, M. Chu, and H. Sun, "Repetitive motion planning of free-floating space manipulators," *Int. J. Adv. Robot. Syst.*, vol. 10, no. 5, 2013,

doi:10.5772/56402.

[16] A. Coelho, H. Singh, K. Kondak, and C. Ott, "Whole-body bilateral teleoperation of a redundant aerial manipulator," in *Proc. IEEE ICRA*, Paris, 2020, pp. 9150–9156, doi:10.1109/ICRA40945.2020.9197028.

[17] T. F. Chan and R. V. Dubey, "A weighted least-norm solution based scheme for avoiding joint limits for redundant joint manipulators," *IEEE Trans. Robot. Autom.*, vol. 11, no. 2, pp. 286–292, Apr. 1995. [entered from memory, check against the paper]

[18] Q. Jia, B. Yuan, and G. Chen, "Representation space analytical method for path planning of free-floating space manipulators," *IEEE Access*, vol. 7, pp. 54228–54251, 2019. [abstract only]

[19] Y. Umetani and K. Yoshida, "Workspace and manipulability analysis of space manipulator," *Trans. SICE*, vol. E-1, no. 1, pp. 116–123, 2001. [not read in full]

[20] S. Cocuzza, M. Artusi, and S. Debei, "Zero reaction workspace of a space manipulator," in *Proc. 61st Int. Astronaut. Congr.*, Prague, 2010, Paper IAC-10.D3.3.9. [author list differs between records, abstract not found]

[21] S. Cocuzza and A. Correnti, "Increased performance reaction control in the operations of hyper-redundant space manipulators," in *Proc. 64th Int. Astronaut. Congr.*, Beijing, 2013, Paper IAC-13.D3.3.4. [abstract only]

[22] M. Pietras, F. Rems, and U. Walter, "Real-time evaluation of attitude constrains during space robot operations," in *Proc. 61st Int. Astronaut. Congr.*, Prague, 2010. [abstract only. A second record with a near-identical abstract lists Pietras, Fleischner and Walter, "Autonomous control of operative constrains during real-time teleoperation of space robots," pp. 8344–8350]

[23] D. N. Nenchev and K. Yoshida, "Singularity-consistent teleoperation techniques for redundant free-flying robots," in *Proc. AIAA Guid. Navig. Control Conf.*, Portland, Aug. 1999, Paper AIAA 1999-4305, doi:10.2514/6.1999-4305. [abstract only]

[24] S. Hauck, D. Abdlkarim, J. Dudley, P. O. Kristensson, E. Ofek, and J. Grubert, "ReachVox: Clutter-free reachability visualization for robot motion planning in virtual reality," in *Proc. IEEE ISMAR*, 2025, arXiv:2508.11426. [abstract only]

[check every reference against the paper itself before submission: volume, issue, pages]